DIVE-HMM / CCBF — Handoff for autonomous continuation

Written for a FRESH chat window that must continue this research autonomously while the user sleeps. Read this fully before acting. Everything you need persists in this project folder `C:\Users\ADMIN\Desktop\HMM-Develop\` *and in the machine's Python env +* `~/.kaggle/access_token`.

---

0. YOUR MISSION (autonomous mode)

Continue the research loop **without asking the user for decisions** (they are asleep):

> **Generate hypothesis → build/modify notebook → run on Kaggle (T4) → pull results → critique honestly → iterate**, to find a **genuinely strong, A-worthy contribution* for a NeurIPS paper on visual place recognition / localization under appearance change.

Operational rules (the user set these): 1. **Default GPU = T4** (tier `t4`) — allocated instantly, no queue. Use for ALL iteration/smoke/dev. 2. **RTX PRO 6000** (tier `rtx`) — ONLY for large-scale (full 143k Nordland) or large-model runs. 3. **Stall rule:** if a kernel shows **no output for ~10 minutes**, it is queue-stuck (happens with `rtx` when the GPU is busy). **Auto-cancel and re-push a fresh run** (for `rtx`, consider dropping to `t4`, or just retry). Don't wait indefinitely. 4. Keep working autonomously; report progress but don't block on the user. 5. **Be scientifically honest** — the current idea (CCBF) is **NOT yet proven** (see §5). Your job is to either make it work on a setting where it genuinely helps, or pivot to a better contribution. Do not fabricate or oversell results.

---

1. PROJECT & RESEARCH IDEA

Domain: Visual Place Recognition (VPR) / visual localization under extreme appearance change (seasons, weather, day/night). Target: **NeurIPS A***. The user's workflow is in `Research Workflow (1).pdf`; datasets/env in `hmm_datasets_environment_summary(1).pdf`.

Current idea — CCBF (Calibrated Condition-Belief Filtering): a lightweight, descriptor-agnostic probabilistic layer that runs a **factored Bayes filter over (place p_t, condition c_t)**. The condition is a latent appearance state tracked online (label-free) with a sticky prior; the inferred condition selects a **calibrated, condition-conditional observation likelihood** `p(d|p,c)` (per-condition temperature τ_c). Full method + proofs in **`CCBF_methodology.md`** (problem formulation, factored HMM, density-ratio/temperature emission, Rao-Blackwellized filter, Theorem 1 "miscalibration propagation", Theorem 2 "CCBF consistency/calibration").

Headline framing (decided): NOT recall (Nordland is saturated for recall with strong descriptors) but **calibration (ECE/reliability) + hard conditions + open-set/off-map**. Nordland = controlled multi-condition testbed.

Idea tournament (Phase 2), in case you pivot: A = CCBF (chosen), B = Evidential Sequence VPR (Dirichlet evidence + proper scoring, uncertainty-weighted belief propagation), C = Online Condition-Matched Retrieval. If CCBF keeps failing, B or a new idea may be the contribution.

---

2. DELIVERABLES & FILE LAYOUT (all persistent in the project folder)

File / folder	What it is
`CCBF_methodology.md`	Method + Theory (paper-ready, 4 theorems with proofs).
`DIVE_HMM_CCBF_NB1.ipynb`	NB1 v3 — foundation + core CCBF + smoke. **SALAD works** (validated on T4).
`DIVE_HMM_CCBF_NB2.ipynb`	NB2 — headline experiments (4-season Nordland + RobotCar/CMU probe). **Ran on T4**, results in §5.
`nb_src/`, `nb2_src/`, `diag_src/`	The `.src` cell-source files the notebooks are built from (edit these, then rebuild).
`automation/`	Kaggle automation scripts + local test harnesses (see §3).
`HANDOFF_CCBF.md` / `.pdf`	This document.
`c-hi-u-folder-n-y-cheeky-kay.md` (in `~/.claude/plans/`)	Original approved plan.

Notebooks are built from `.src` files, NOT edited directly. Each `.src` holds cells separated by `#%%CODE` / `#%%MARKDOWN` markers. Build with `automation/build_nb.py <out.ipynb> <src_dir>` (compiles every code cell +

validates).

---

3. KAGGLE AUTOMATION (how to run things)

Auth: already configured — token at `~/.kaggle/access_token` (Kaggle CLI 2.2.5 reads it, introspects username `nhatlong1103`). Python deps installed in the machine env (`numpy`, `kaggle`, `python-dateutil`, `python-slugify`, `tqdm`, `protobuf`, `matplotlib`). If a dep is missing: `python -m pip install <dep>`.

Scripts in `automation/`: - `build_nb.py <out.ipynb> <src_dir>` — assembles + compile-checks a notebook from `.src` files. - `make_meta.py <dir> <id> <code_file> <comma-datasets> [tier]` — writes `kernel-metadata.json`. **tier `t4`** (default, instant) or **tier `rtx`** (RTX PRO 6000 — adds `machine_shape=NvidiaRtxPro6000` + `competition_sources=[arc-prize-2026-arc-agi-3]` + the byod `docker_image`; this exact combo is what unlocks RTX — machine_shape alone silently falls back to T4). - `kaggle_run.py <kernel_dir> <out_dir>` — push + poll status + pull output. Run in background. - `poll_pull.py <user/slug> <out_dir>` — poll + pull an already-pushed kernel (no push). - `gpu_watch.py <user/slug> <tmp_dir>` — poll the partial log until the GPU-name line appears (early detection). - `test_core.py` / `test_core2.py` — **local** runtime tests of the numpy core of NB1 / NB2 (no torch/Kaggle needed; use these to catch logic bugs before pushing — they already caught real bugs).

Standard loop to run a notebook:

```
cd automation            # scripts live here
python build_nb.py "../DIVE_HMM_CCBF_NB2.ipynb" ../nb2_src   # (adjust src dir)
python test_core2.py                                          # local logic check
mkdir -p run/nb2 && cp "../DIVE_HMM_CCBF_NB2.ipynb" run/nb2/nb2.ipynb
python make_meta.py run/nb2 nhatlong1103/dive-hmm-ccbf-nb2 nb2.ipynb \
   nhatlong1103/bundle,nhatlong1103/nordland-full-47-6gb,nhatlong1103/robotcar-seasons-v2,nhatlong1103/extend
python kaggle_run.py run/nb2 run/nb2_out      # run in BACKGROUND; notifies on completion
```

Pull & read a finished kernel's stdout (results are a JSON log of `{stream_name,data}` records):

```
kaggle kernels output nhatlong1103/<slug> -p <dir>
python -c "import json,glob;t=open(glob.glob('<dir>/*.log')[0],encoding='utf-8').read();
recs=json.loads(t);print(''.join(r['data'] for r in recs if r.get('stream_name')=='stdout'))"
```

Dataset slugs (owner `nhatlong1103`): `bundle` (models: SALAD/DINOv2/SuperPoint/LightGlue/pycolmap + checkpoints), `nordland-full-47-6gb` (4 seasons, 143k), `norlandshort` (2-season smoke), `robotcar-seasons-v2`, `extended-cmu-seasons`, `extended-cmu-seasons-2`, `4seasons`. Also `yuan1103/robotcar-location-metadata` (RobotCar GT metadata — may help NB3).

Kernel slugs already created: `dive-hmm-diag` (env/SALAD-source probe), `dive-hmm-ccbf-nb1`, `dive-hmm-ccbf-nb2`. Pushing re-versions them.

Env facts (from the diag run): torch 2.11.0+cu128, T4 = 15.6 GB, bf16 supported. **Throughput ~20 img/s** (SALAD/DINOv2 fwd 21/s, decode 16/s) → smoke (≈1–3k imgs) ~3–7 min; full 143k on T4 ≈ 2 h (use `rtx`, ~5–10× faster).

---

4. THE CORRECT SALAD LOADER (already in the notebooks)

SALAD (serizba/salad, CVPR'24) loads from the bundle by building backbone+aggregator directly via `helper.get_*` (bypassing `VPRModel`/Lightning), with the released configs:

```
sys.path.insert(0, f"{bundle}/sources/salad")
from models import helper
backbone   = helper.get_backbone("dinov2_vitb14",
   {"num_trainable_blocks":4,"return_token":True,"norm_layer":True})   # loads DINOv2 offline itself
aggregator = helper.get_aggregator("SALAD",
   {"num_channels":768,"num_clusters":64,"cluster_dim":128,"token_dim":256})
sd = torch.load(f"{bundle}/checkpoints/dino_salad.ckpt",map_location="cpu"); sd=sd.get("state_dict",sd)
backbone.load_state_dict({k[9:]:v for k,v in sd.items() if k.startswith("backbone.")},strict=False)
aggregator.load_state_dict({k[11:]:v for k,v in sd.items() if k.startswith("aggregator.")},strict=False)
# forward: aggregator(backbone(x)) -> [B, 8448] L2-normalized descriptor
```

Verified: `backbone(missing=0,unexpected=0) agg(missing=0,unexpected=0)`, dim **8448**.

---

5. RESULTS SO FAR — AND THE HONEST VERDICT

NB1 v3 (T4, SALAD, Nordland Short summer→winter, 600 frames): SALAD loads perfectly; single-image R@1 **0.617** (vs 0.27 for the earlier buggy DINOv2 mean-pool → 2.3× better). Pipeline PASS. CCBF=fixed (single-condition episode, expected).

NB2 (T4, SALAD, Nordland FULL 4 seasons, 800 frames/season aligned; ref=summer, queries=spring/fall/winter):

Method	R@1	ECE
single_image	0.819	0.076
fixed_filter	**0.889**	0.028
ccbf	0.884	0.030

- **ECE-vs-T:** fixed `[.085,.03,.026,.016,.033,.021,.019,.037,.036,.026]`, ccbf `[.052,.037,.026,.019,.025,.019,.022,.031,.033,.026]` — essentially overlapping; **no "fixed rises, CCBF flat" signature**.
- **Off-map AUROC:** ccbf_null 0.992, fixed_null 0.993, SUE 0.935 (filter null >> single-image uncertainty, but CCBF≈fixed).
- **Label-free condition tracking: 0.974** (4 conditions, chance 0.25) — works great.

■■ THE CRITICAL FINDING (read carefully)

CCBF ≈ fixed_filter on every metric. Root cause: the fitted **per-condition temperatures are nearly identical** (spring/summer/fall τ=0.0309, winter τ=0.0253). **SALAD is so condition-robust on Nordland that there is almost no condition-dependent miscalibration to exploit** — so condition-conditional calibration (CCBF's whole point) ≈ global calibration (fixed). Theorem 1's empirical signature does not appear because the premise (non-negligible per-condition potential-error variance) barely holds here.

*Conclusion: CCBF is NOT yet an A contribution on Nordland+SALAD.** The temporal filter helps recall (0.82→0.89) but the *condition* machinery adds nothing measurable here.

■ THE MAKE-OR-BREAK NEXT EXPERIMENT

Test whether per-condition temperatures differ SUBSTANTIALLY on RobotCar Seasons day→night/rain. Hypothesis: with a huge day↔night appearance gap, SALAD's match/non-match score distributions differ drastically by condition → τ_night ■ τ_day → CCBF's condition-conditional calibration should yield real ECE gains and the ECE-vs-T separation Theorem 1 predicts. **This single experiment largely decides whether CCBF lives or must be replaced.**

If CCBF still ≈ fixed on RobotCar hard conditions → **pivot**: either (a) reframe the contribution around the two things that DID work (excellent label-free condition tracking 0.97 + calibrated off-map detection beating SUE), (b) switch to tournament idea B (Evidential), or (c) generate a new idea. Be honest; don't force a weak idea.

---

6. AUTONOMOUS AGENDA (do these in order)

1. **RobotCar per-condition calibration diagnostic (TOP PRIORITY).** Build a small T4 notebook: extract SALAD descriptors for RobotCar train images per condition (format in §7), define matches by pose proximity, and **fit/plot τ_c per condition (day/dusk/night/night-rain/snow/rain/sun)**. If τ_night ■ τ_day → CCBF has a real home; build the sequence experiment there. If τ's are similar → pivot.
2. **Fix RobotCar/CMU GT loaders** (§7) and run CCBF vs baselines on RobotCar night (recall + ECE + off-map), the paper's hard-condition headline.
3. **Ablations** once a win exists: K (conditions), calibrated vs raw likelihood, sticky ρ, smoothing, null state.
4. **Scale** the winning setting to full data on **tier `rtx`**.
5. **Write Intro + Related Work + Contributions** (Method+Theory already done) once a real contribution is confirmed.

Keep every run on **T4**; use `test_core*.py` locally before each push; one hypothesis per iteration; log the honest verdict each time.

---

7. GOTCHAS & EXACT FORMATS (will save you hours)

- **RobotCar `robotcar_v2_train.txt` format:** `"<cond>/<cam>/<ts>.jpg"` then a **flattened 4×4 pose matrix** (12 floats, row-major). **Translation = indices [3,7,11]** of the 12 floats (NOT quaternion+xyz). The NB2 probe wrongly used `nums[4:7]` — fix it. The file contains **query conditions only** (rain/night/dawn/night-rain/snow/sun/overcast-summer/dusk/overcast-winter, ~200 imgs each); **`overcast-reference` is**

NOT in train.txt (reference poses come from the 3D model `3D-models/all.nvm` / `.db`). For a self-contained eval, use the train images across conditions with pose-GT, or get `overcast-reference` poses from the NVM.

- **CMU `camera-poses/*` format:** `"img_XXXXX_cN_<ts>us.jpg" qw qx qy qz tx ty tz` → **xyz = last 3 floats**. Each slice has `database/` + `query/` images and ~12 pose files. (NB2 probe's `nums[4:7]` is off by one; use `nums[-3:]`.)
- **numpy 2.x:** `float(arr)` fails for a size-1 but ndim≥1 array — use `float(arr.reshape(()))`. (Already fixed in the code; watch for new cases.)
- **`dir()` inside a function** returns locals — use `"X" in globals()` to test module-level names. (Already fixed.)
- **GeM pooling needs non-negative inputs** — DINOv2 tokens are signed; use CLS+mean, not GeM-with-clamp (that collapsed descriptors to a narrow cone — the original NB1 bug). SALAD is the real descriptor anyway (dim 8448).
- **RTX tier queues:** `tier=rtx` can sit with 0 output if the Blackwell GPU is busy; apply the 10-min stall rule (cancel + retry, or use T4). `kaggle kernels status <slug>` shows `KernelWorkerStatus.RUNNING/COMPLETE/CANCEL_ACKNOWLEDGED`; parse with substring `.complete`/`.error`/`.cancel` (NOT a regex that grabs the enum class name — that bug made the poller wait to timeout).
- **Kaggle log parsing:** output is a JSON array of `{stream_name,data,time}`; concatenate `data` where `stream_name=='stdout'`.
- **Mount size:** attach only the datasets a run needs; the user's `ccbf-hmm` kernel shows all of them can be attached together if needed.

---

8. SCIENTIFIC GUARDRAILS

Follow the user's `Research Workflow`: loop **Generate** → **Critique** → **Verify** → **Improve** → **Re-evaluate**; don't defend a weak idea. A* gate = novelty + technical depth + significance + experimental verifiability + generalizability + reviewer-robustness. **The current honest state: CCBF's novelty is clear but its empirical value is unproven — the RobotCar hard-condition test is decisive.** Report numbers as they are (including CCBF≈fixed where true). The two components that already work (label-free condition tracking; calibrated off-map > SUE) are assets worth keeping in whatever final story emerges.